Software Engineering - 22AIE311

Software's Dual Role

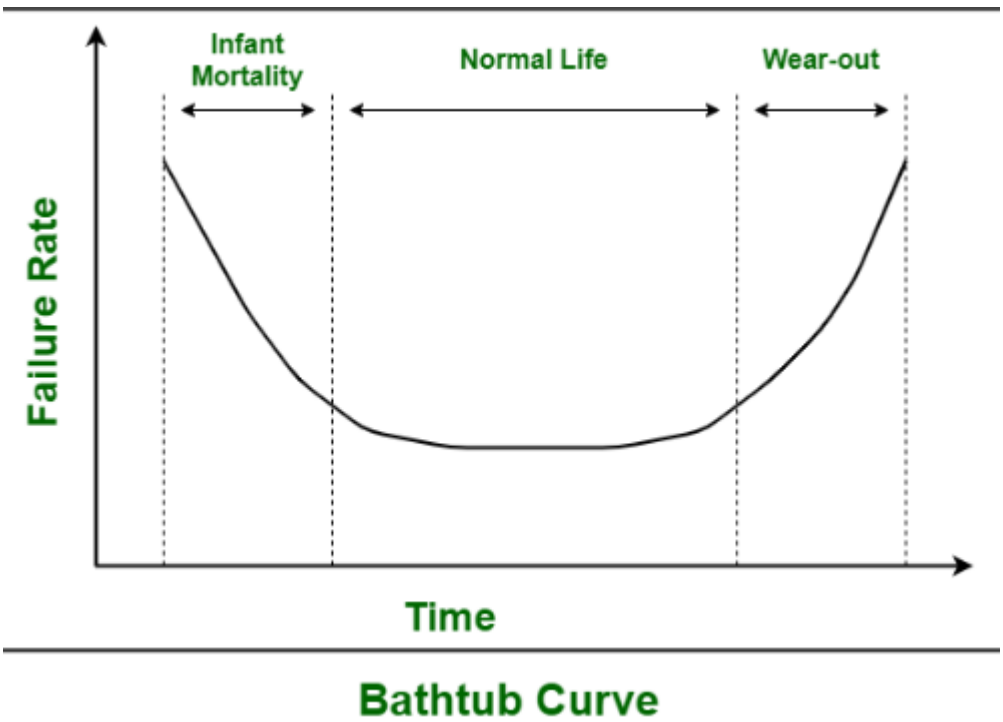
1. **Software is a product**

- Delivers computing potential
- Produces, manages, acquires, modifies, displays and transmits information

2. **Software is a vehicle for delivering a product**

- Supports or directly provides system functionality
- Controls other programs (e.g., an operating system)
- Effects communications (e.g., networking software)
- Helps build other software (e.g., software tools)

Bathtub Curve



Bathtub Curve is a graph that is used to graphically demonstrate run-to-failure maintenance strategy.This curve represents overall life cycle of assets and failure rate of overall population of assets over time. Bathtub curve is divided into three different sections:

1. **Infant Mortality Section :** In this section, asset is beginning with its usage for first time. Initially, failure rate is very high and with increasing time, there is a gradual decrease in failure rate. Failures are usually occurred due to manufacturing defects, installation issues, design issues, material defects, etc.
2. **Normal Life Section :** This section represents normal operating life of assets.In this section, asset is still experiencing failure but at normal and low rate. Failures are usually occurred due to overloading, hidden defects, collision with other objects, mistakes of personnel, etc.

3. **Wear-out Section :** In this section, there is gradual increase in failure rate of assets with increasing time. Failures are usually occurred due to fatigue, wear, gradual deterioration, corrosion, etc. This period represents end of life cycle of assets.

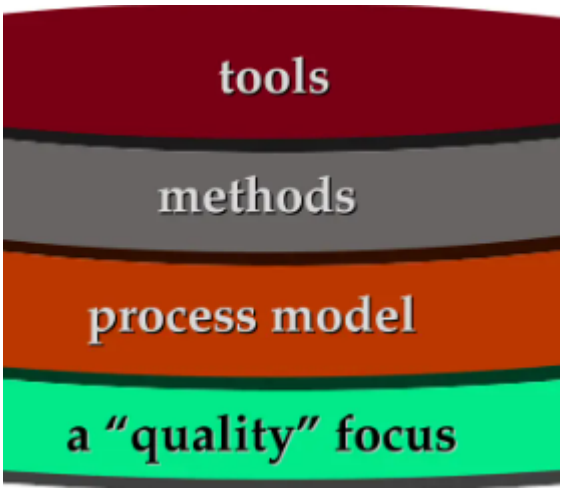
Hardware vs Software

Parameters	Hardware	Software
Basic Definition	Hardware is a physical part of the computer that causes the processing of data.	Software is a set of instructions that tells a computer exactly what to do.
Development	It is manufactured.	It is developed and engineered.
Dependency	Hardware cannot perform any task without software.	The software can not be executed without hardware.
Process of creating	Electronic and other materials are used to create hardware.	Created by utilizing a computer language to write instructions.
Tangible	Hardware is tangible as hardware is a physical electronic device, that can be touched.	Software is intangible as we can see and also use the software but can't touch them.
Durability	Hardware typically wears out over time.	The software does not wear out with time. However, it may contain flaws and glitches.

Legacy Software

- The older programs which are developed decades ago that are still in use by performing modifications in order to meet the business requirements.
- Many legacy systems remain supportive to core business functions and are important to business. Hence, legacy software is characterized by longevity and business criticality.

Software as a Layered Technology



- Any engineering approach must rest on an organizational commitment to **quality**
- **Process** is the glue that holds the technology layers together & enables timely development of computer software. It forms the basis for management control of software projects
- **Methods** encompass a broad array of tasks that include communication,requirements analysis, design modeling, program construction,testing support
- **Tools** provide automated or semi-automated support for the process and methods

SDLC Models

Feature	Waterfall	V-Model	Incremental	Prototyping	RAD (Rapid App Dev)	Spiral
Approach	Linear, sequential phases.	Sequential, with parallel verification/validation.	Iterative, delivering functional increments.	Iterative, building and refining prototypes.	Iterative, focus on quick prototypes & user feedback.	Iterative, risk-driven cycles (spirals).
Requirements	Must be clear and fixed upfront.	Stable, well-defined requirements.	Evolve, delivered in stages.	Unclear, complex, or changing.	Evolving, user-driven needs.	Large, complex, high-risk projects with changing needs.
Flexibility	Very Low; difficult to change.	Low; changes are costly.	Medium; easy to adapt increments.	High; easy to incorporate feedback.	High; rapid changes possible.	High; risk-focused changes.
User Involvement	Only at start & end.	Limited, during verification.	Intermediate, reviews each increment.	High; continuous feedback.	Very High; throughout the cycle.	High; frequent feedback.
Risk Mgmt	Poor; high risk.	Better than Waterfall (early testing).	Medium; risks managed per increment.	Good; early risk identification.	Medium; quick feedback helps.	Excellent; central focus.
Best For	Small projects, fixed scope.	Projects needing strict testing (medical).	Quick delivery of core features.	Unclear requirements, complex systems.	Fast-changing, user-centric apps.	Large, complex, high-risk systems.

When to Use Specific Models

W Waterfall Model

BEST FOR Small projects with **clear, unchanging requirements** and limited user feedback needed until the end.

WHY Simple, structured, easy to manage, but inflexible to changes.

V V-Model (Verification & Validation)

BEST FOR Projects where quality and testing are paramount, like safety-critical systems — ensuring each development phase has a corresponding test phase.

WHY Strong emphasis on testing early and thoroughly throughout the lifecycle.

S Spiral Model

BEST FOR Large, complex, high-risk projects where risks need careful management through prototyping in each loop.

WHY Combines iterative development with risk analysis, making it flexible for complex endeavors.

I Incremental / Iterative Model

BEST FOR Large systems that can be broken down, delivering working pieces (increments) early and continuously refining them.

WHY Early user involvement, easier to manage complexity, provides early value.

R RAD (Rapid Application Development)

BEST FOR Projects needing very quick delivery (tight deadlines) and where user requirements are well-understood but rapid prototyping is key.

WHY Focuses on quick prototyping and user feedback to speed up development.

A Agile (Scrum / Kanban)

BEST FOR Projects where requirements are expected to **change**, need rapid delivery, and require continuous customer collaboration.

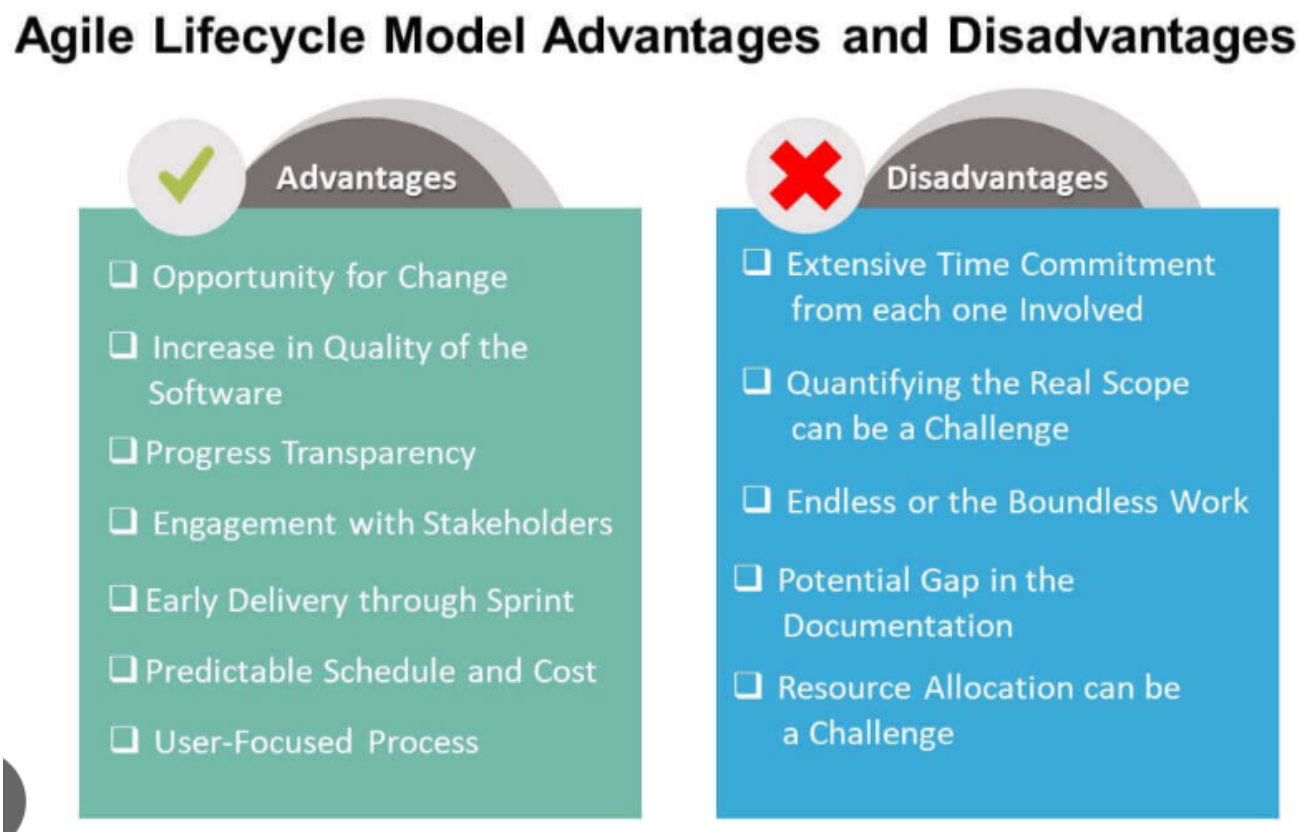
WHY Highly adaptable, low cost for changes, fast feedback loops.

Traditional vs Agile

Characteristics	Agile approach	Traditional approach
Organizational structure	Iterative	Linear
Scale of projects	Small and medium scale	Large-scale
User requirements	Interactive input	Clearly defined before implementation
Involvement of clients	High	Low
Development model	Evolutionary delivery	Life cycle
Customer involvement	Customers are involved from the time work is being performed	Customers get involved early in the project but not once the execution has started

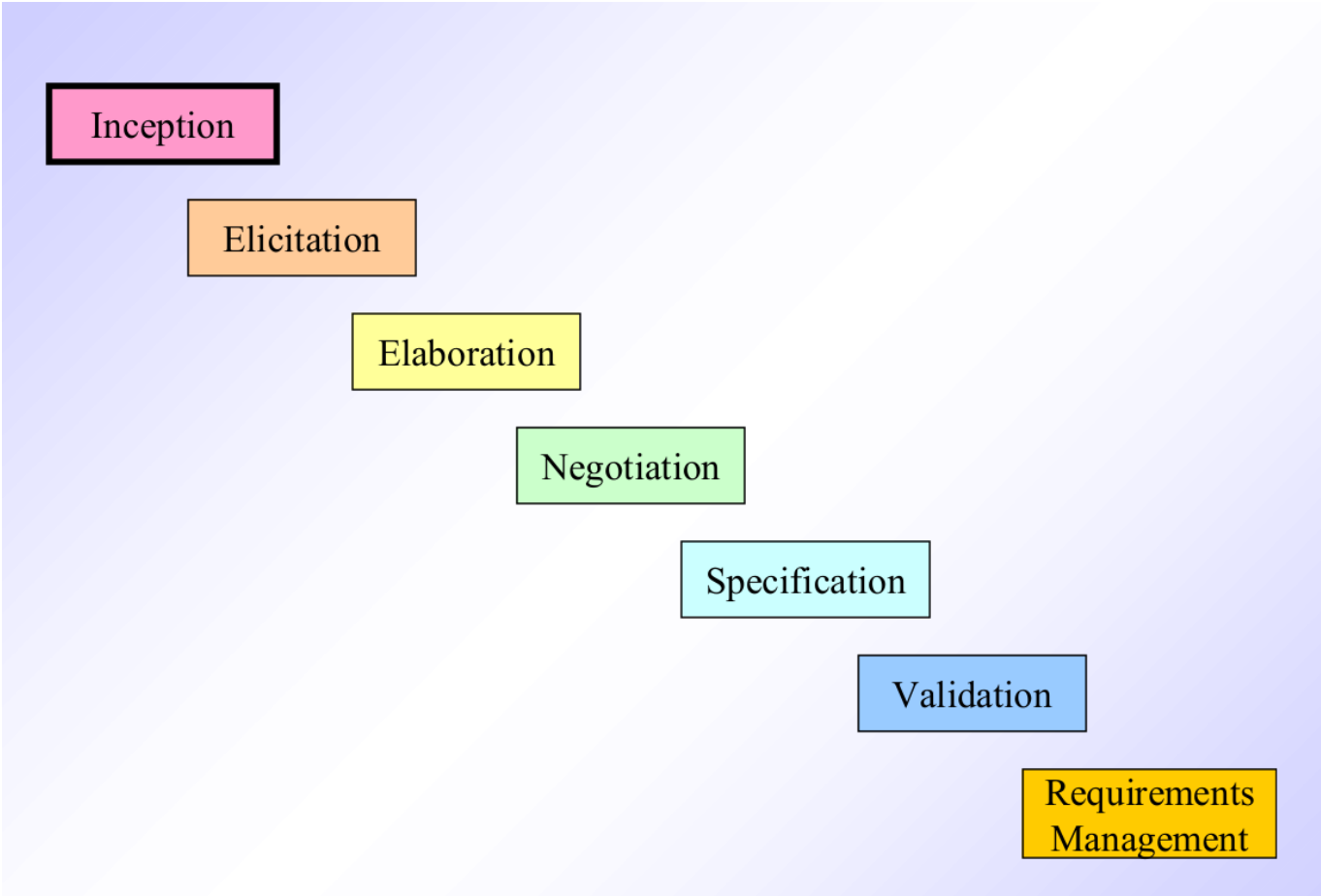
Agile Scrum Methodology

- **Product owner:** Creates an estimated wish list that is identified as a **product backlog**.
- **Product Backlog** is a list of new features, enhancements, bug fixes, tasks, or work requirements needed to build a product.
- **Scrum team:** It takes one little part of the top wish list, termed as **Sprint Backlog** and work out in order to implement it.
- **Sprint Backlog** is a set of product backlog tasks that have been promoted to be developed during the next product increment.
- **Product Increment** is the customer deliverables that were produced by completing product backlog tasks during a sprint.
- Scrum team concludes their **sprint** backlog task in a Sprint, i.e., a period of 2-4 weeks
- The progress of their work can be accessed through a meeting that is called **Daily Scrum**.
- The **Scrum Master** maintains the team focused towardtheir targets



Requirement Engineering

It is the systematic process of defining, documenting, and maintaining software system requirements. It builds a bridge from the system requirements into software design and construction



1. Inception Task

- A basic understanding of the problem
- Identify the stakeholders
- Recognize multiple viewpoints
- Work towards collaboration
- Break the ice and initiate the communication

2. Elicitation Task

It is difficult because of problems of scope, understanding and volatility

It is accomplished through two activities:

1. **Collaborative requirements gathering :** The goal is to identify the problem, propose elements of the solution, negotiate and specify a preliminary set of solutions
2. **Quality Function Deployment:** It identifies three types of requirements:

- **Normal requirements** - Requirements stated during meetings with the customer
- **Expected requirements** - Fundamental requirements that may not be explicitly stated by customers
- **Exciting requirements** - Requirements that go beyond the customer's expectations

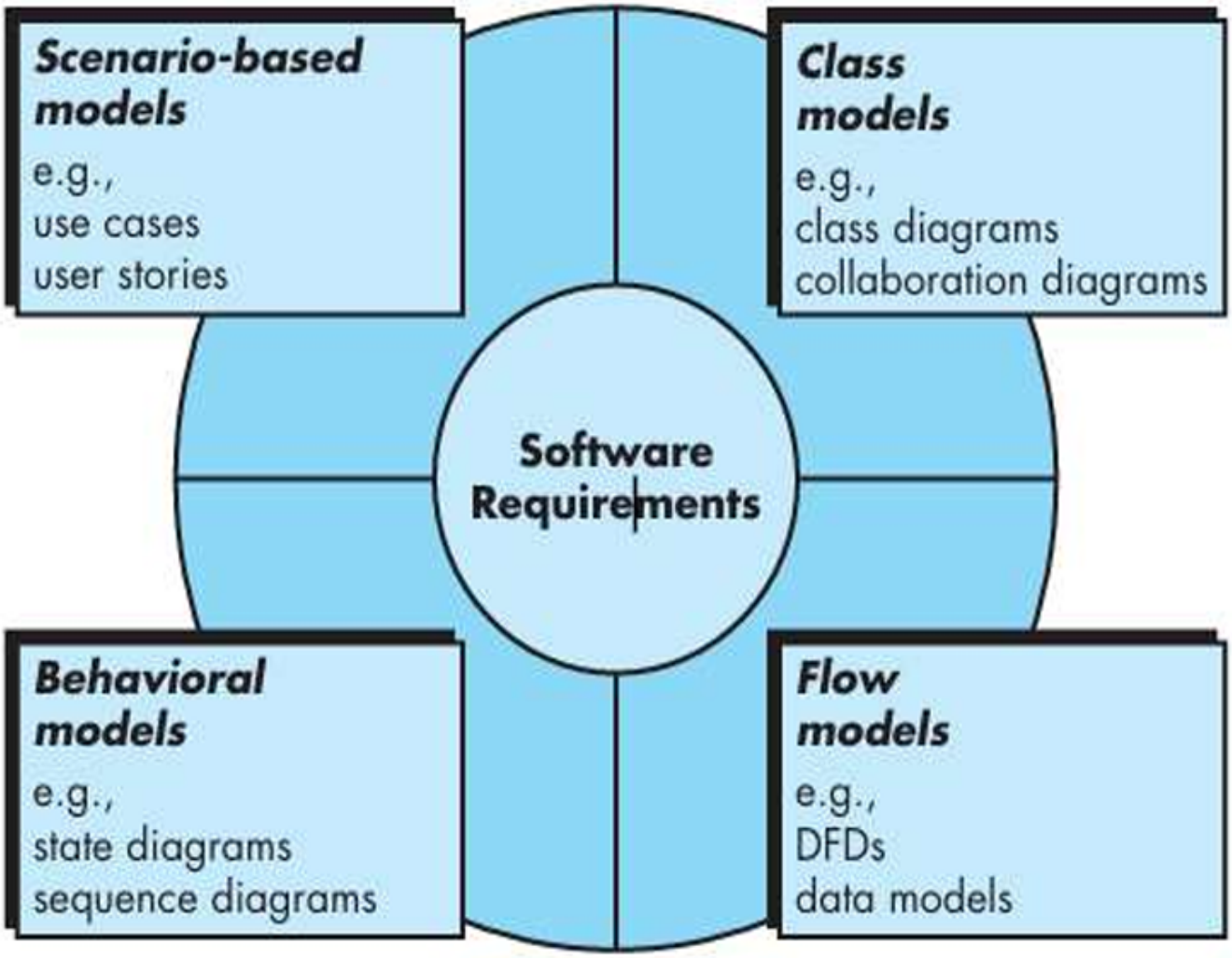
3. Elaboration Task

- It is an analysis modeling task
- Use cases are developed
- The end result includes the functional, informational, and behavioral domains of the problem

https://www.geeksforgeeks.org/system-design/use-case-diagram/

Building the Analysis Model

- An analysismodel in software engineering is created to define the functional, behavioral, and datarequirements of a system.
- It acts as a bridge between system description and design.



{width=100% height=50%}

Elements of the analysis model:

- **Scenario-based elements** - Functional, Use Case
- **Class-based elements** - Implied by scenarios
- **Behavioral elements** - State diagram
- **Flow-oriented elements** - Data flow diagram

https://www.geeksforgeeks.org/system-designunified-modeling-language-uml-class-diagrams/

4. Negotiation Task

- Requirements are **prioritized** by the customers, users, and other stakeholders
- Risks associated with each requirement are identified and analyzed

5. Specification Task

- It is the **final work product** produced by the requirements engineer
- It formalizes the informational, functional, and behavioral requirements of the proposed software

6. Validation Task

This process checks if the software documentation is consistent, unambiguous, achievable, testable, necessity etc.

7. Requirements Management

- The project team performs a set of activities to identify, control, and track requirements
- A **Requirement Traceability Matrix** (RTM) is a document that tracks project requirements
- RTM is a comprehensive, dynamic document that tracks evolving project requirements.
- It ensures the final product aligns with initial user expectations.

RTM is of 3 types:

1. **Forward traceability:** Objective: to ensure every requirement is tested and properly implemented. Direction: from requirements to test cases/deliverables

2. **Backward traceability / Reverse Traceability:** Objective: to ensure no unnecessary features are added and to verify alignment with initial requirements. Direction: from test cases / deliverables to requirements.

3. **Bidirectional traceability:** Objective: to ensure complete alignment between requirements and test cases, supporting thorough verification and impact analysis. Direction: from / to requirements and from / to test cases / deliverables.

Software Requirements Specification (SRS)

- It lays out functional and non functional requirements and may include a set of use cases
- The output of requirement engineering is the SRS
- First SRS should be written by the **customer**
- Second, SRS should be written by the **developer**
- Quality characteristics of a good SRS are: Complete, Consistent, Unambiguous, Modifiable, Relevant, Human-readable etc

Functional Requirements

- Functional requirements describe **what** the software should do
- Example : User Authentication, Search Functionality, Explanation etc
- They are gathered by Interviews, Surveys and Workshops

Non-functional Requirements

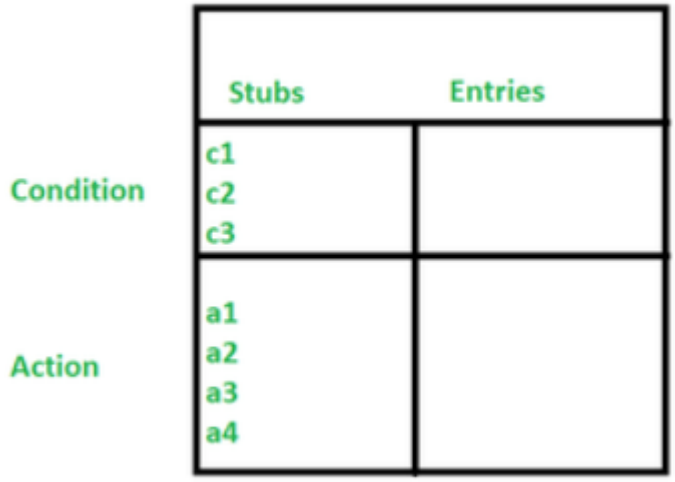
- Non-functional requirements describe **how** the software performs a task rather than what it should do
- Example : Performance, Usability, Reliability, Security etc
- Two main categories: **Execution qualities** - security and usability, which are observable at run time, **Evolution qualities** - testability, maintainability, and scalability that are present in the static structure of the software system.
- They are gathered by Performance Benchmarks, Security Standards and Usability Testing.

FUNCTIONAL vs NONFUNCTIONAL REQUIREMENTS

	Functional requirements	Nonfunctional requirements
Objective	Describe what the product does	Describe how the product works
End result	Define product features	Define product properties
Focus	Focus on user requirements	Focus on user expectations
Essentiality	They are mandatory	They are not mandatory but desirable
Origin type	Usually defined by the user	Usually defined by developers or other tech experts
Testing	Component, API, UI testing, etc. Tested before nonfunctional testing	Performance, usability, security testing, etc. Tested after functional testing
Types	Authentication, authorization levels, data processing, reporting, etc.	Usability, reliability, scalability, performance, etc.

Decision Table

- They are **tabular representations** of the possible combinations of conditions and actions that determine the behavior of a system
- They consist of four quadrants:
- **Condition stubs** - variables - input data, user roles
- **Condition entries** - values - yes/no, numeric
- **Action stubs** - outcomes - updating a database, sending an email
- **Action entries** - indicators - X, O, or blank
- Use of Decision Table in Requirements Engineering: Clarifying Business Rules, Handling Complex Decision Logic, Documentation, Test Case Generation
- They are helpful in testing, development process, prepare requirements etc



Decision Tree

- It is a **graphical representation** of decision-making
- It helps visualize how different conditions lead to specific outcomes
- Use of Decision Tree in Requirements Engineering: Visualizing Decision Logic, Defining System Behavior, Providing Traceability, Simplifying Complex Decisions

---

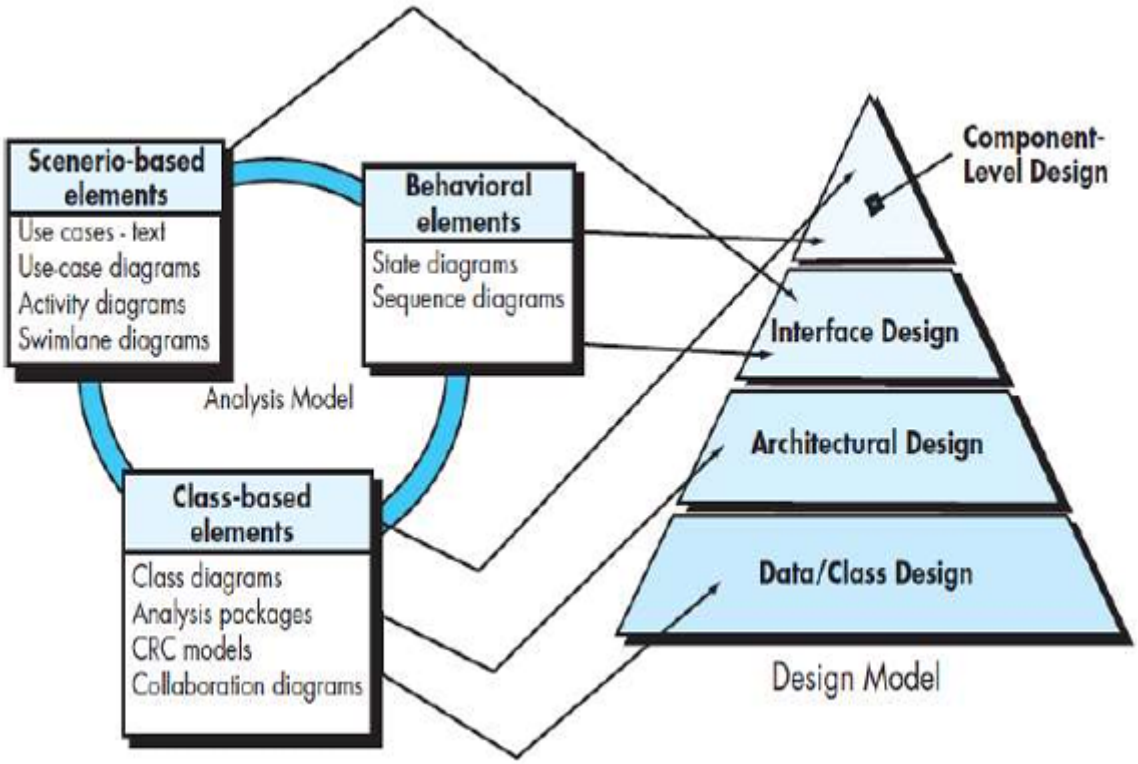
Requirements Analysis

- It specifies software's operational characteristics, indicates software's interface and establishes constraints that software must meet
- Requirements analysis allows the software engineer (analyst) to :
    - **elaborate** on basic requirements established during earlier requirement engineering tasks
    - **build models** that depict user scenarios, functional activities and problem classes

---

Design Engineering

Software Design Process



McGlaughlin's 3 characteristics (goals) as a guide for the evaluation of a good design are as follows:

1. **Implement All Requirements:** The design must meet all explicit and implicit stakeholder requirements.
2. **Ensure Readability:** The design must be easily understood by developers, testers, and maintainers.
3. **Provide a Complete Picture:** The design should fully address the data, functional, and behavioral aspects of the system.

HP's set of software quality attributes as acronym **FURPS**

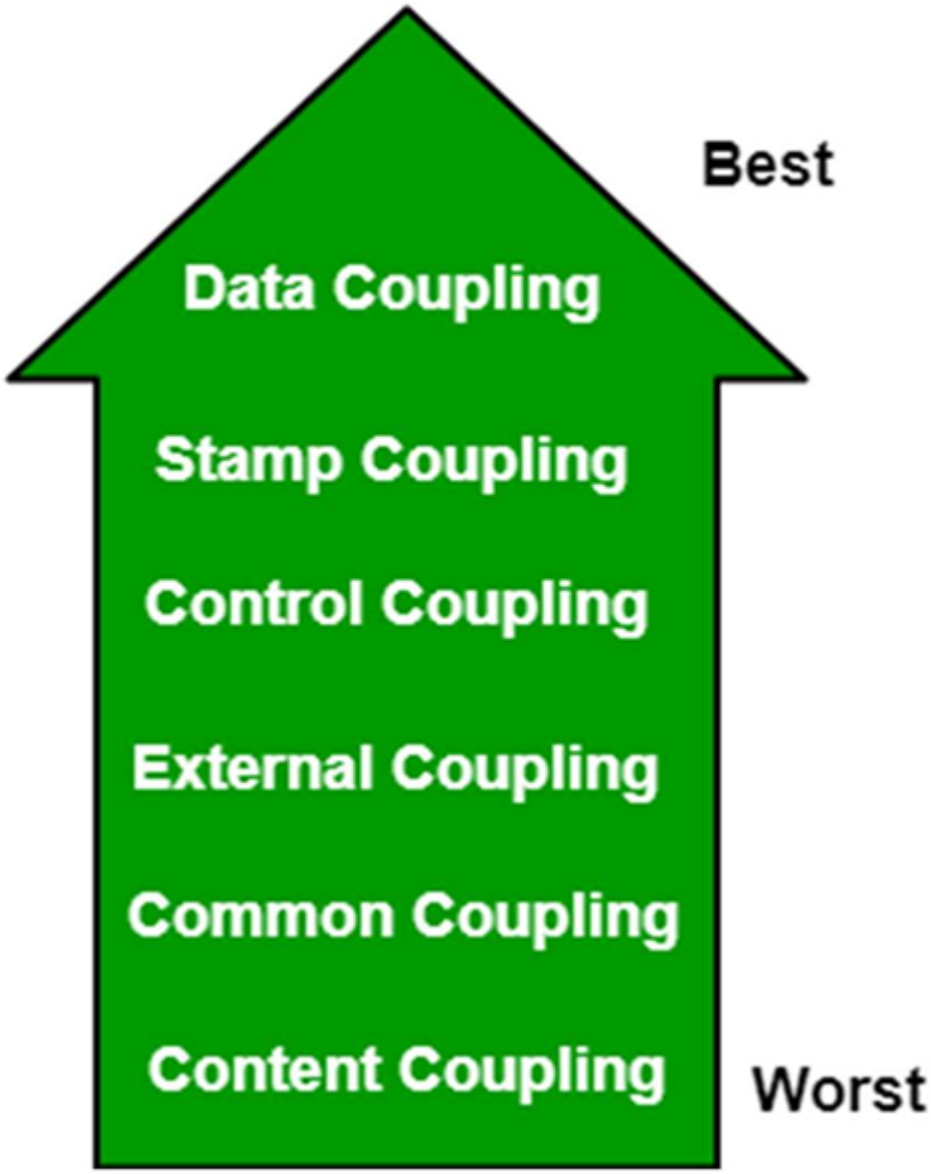
1. **Functionality:** Refers to the feature set, capabilities, and security of the application.
2. **Usability:** Focuses on human factors, aesthetics, consistency in the user interface, online help, and documentation.
3. **Reliability:** Covers the frequency of failure, recoverability, predictability, accuracy, and mean time between failures (MTBF).
4. **Performance:** Measures the software's efficiency in terms of speed, response time, resource consumption, throughput, and scalability.
5. **Supportability:** Encompasses testability, maintainability, serviceability, flexibility, installability, and localizability.

Design Concepts

"The beginning of wisdom for a [software engineer] is to recognize the difference between getting a program to work, and getting it right." ~ M. A. Jackson

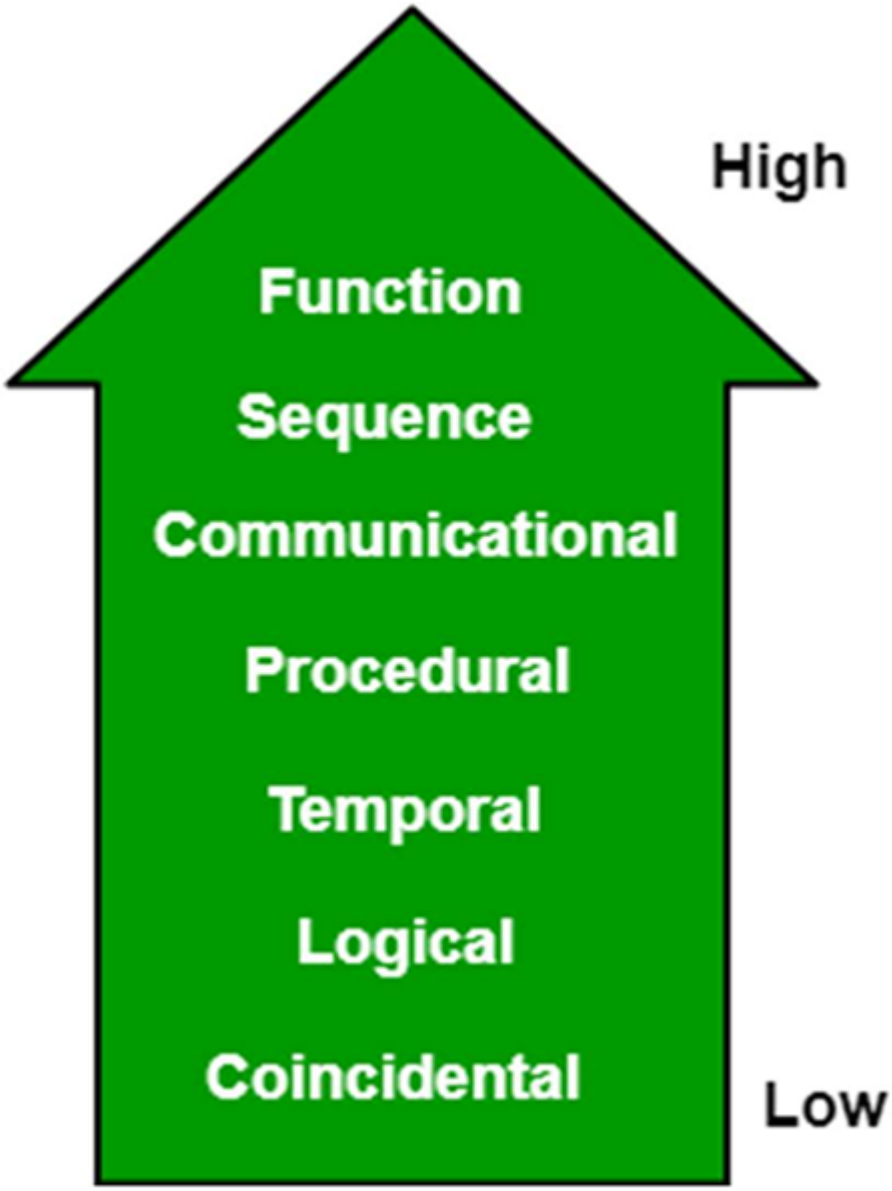
1. **Abstraction :** Abstraction simply means to hide the details to reduce complexity and increase efficiency or quality. **Procedural abstraction** refers to a sequence of instructions that have a specific and limited function. **Data abstraction** a named collection of data that describes a data object.
2. **Modularity:** It means dividing the system or project into smaller parts to reduce the complexity of the system or project. Modularity in design means subdividing a system into smaller parts so that these parts can be created independently and then use these parts in different systems to perform different functions.
3. **Architecture :** It is a concept that focuses on various elements and the data of the structure. These components interact with each other and use the data of the structure in architecture.
4. **Refinement:** It is a concept of software design is a process of developing or presenting the software or system in a detailed manner which means elaborating a system or software.
5. **Refactoring:** It means reconstructing the design to reduce complexity and simplify it without impacting the behavior or its functions.
6. **Pattern:** It means the repetition of a solution to a common recurring problem within a certain context.
7. **Coupling:** It refers to the degree of interdependence between software modules. High coupling means that modules are closely connected and changes in one module may affect other modules. Low coupling means that modules are independent.
8. **Cohesion:** It refers to the degree to which elements within a module work together to fulfill a single, well-defined purpose.

Types of Coupling



1. **Data Coupling:** Modules communicate by passing only data information
2. **Stamp Coupling:** A whole data structure is passed, but only part is used
3. **Control Coupling:** Modules communicate by passing control information
4. **External Coupling:** The modules depend on other modules or external to the software being developed.
5. **Common Coupling:** The modules have shared data such as global data structures.
6. **Content Coupling:** In a content coupling, one module can modify the data of another module, or control flow is passed from one module to the other module.

Types of Cohesion



1. **Coincidental Cohesio:** Unrelated functions are grouped together randomly.
2. **Logical Cohesion:** Related by category, but selected using a control flag.
3. **Temporal Cohesion:** Tasks executed at the same time.
4. **Procedural Cohesion:** Elements are related and must follow a specific order.
5. **Communicational Cohesion:** Functions operate on the same data.
6. **Sequential Cohesion:** Output of one function becomes input to next.
7. **Functional Cohesion:** Module performs exactly one well-defined task.

In conclusion, it’s good for software to have low coupling and high cohesion.

Epics, User Stories and Story Points

- **Epics** provide the big-picture vision, representing significant functionality or features.
- **User stories** break epics down into specific, actionable tasks.
- **Story points** offer a way to measure effort, helping teams estimate and manage their workloads

User Stories

A User Story is a requirement expressed from the perspective of an end-user goal. It is a well-expressed requirement.

The 3C's of User Stories:

1.**Card:**

- The Front of the Card: A unique "Story Identifier", usually a number or reference
- The Back of the Card: Acceptance criteria

2. **Conversation**
3. **Confirmation**

Estimate story points in Scrum

Scrum story points are usually represented using the Fibonacci sequence - 1, 2, 3, 5, 8, 13, 20, 40, and 100

Create a matrix to visualize story point values. Make a row for each number in the Fibonacci sequence.

Story point	Amount of effort required	Amount of time required	Task complexity	Task risk or uncertainty
1	Minimum effort	A few minutes	Little complexity	None
2	Minimum effort	A few hours	Little complexity	None
3	Mild effort	A day	Low complexity	Low
5	Moderate effort	A few days	Medium complexity	Moderate

Backlog Grooming

Makeyourproduct backlog **DEEP**

1. **Detailed Appropriately:** Top items have more detail, clear acceptance criteria, and are ready for development. Lower-priority items are less detailed.
2. **Estimated:** Items are sized to allow for velocity forecasting.
3. **Emergent:** The backlog is a living document, updated continuously as new information and user feedback.
4. **Prioritized:** The backlog is ordered with the most valuable, high-impact, or risk-reducing items at the top.

Extreme Programming(XP)

- Extreme Programming (XP) is an Agile software development methodology that focuses on delivering high-quality software through frequent and continuous feedback, collaboration, and adaptation.

- XP is based on the four simple values:
  - Uniformity
  - Simplicity
  - Communication
  - Feedback and Endurance
- XP is the most suitable for:
  - Small and medium size projects
  - New technologies
  - Projects with unclear requirements
  - Risky projects
- **Pair programming** is a software development practice where two programmers work together on one computer.
- The **driver**, writing code while the other, the **observer or navigator**, reviews each line of code as it's typed.
- This real-time collaboration helps catch errors early, improves code quality

---